

TEST I (2026)

PART I: Introduction to software engineering

TEAM number: 6

(only one solution per team)

Team members (names): Erald Leka, Klajdi Oga, Sibora Muka, Kristiana Jorgo, Erjona Haka

Writer of the answer to TEST I: Erald Leka, Klajdi Oga, Sibora Muka, Kristiana Jorgo, Erjona Haka

For this test you have to build teams. One answer per team is required.

Remark: Most questions of Test I are directly taken from the question part of the respective topic, in particular from multiple-choice questions. This will also be the principle of examinations: Multiple-choice questions from the topics contribute to 50% of the exam questions.

Send your answer to: Klaus Bothe klaus.bothe@hu-berlin.de and Elinda Kajo Mece ekajo@fti.edu.al

Deadline: 23rd March 2026

Give short answers:

For the multiple-choice questions below, *indicate which is true or false (YES/NO) and explain your choice by reference to a slide (like in **sample answer** to 1.1 a) and with some **very short explanation**.*

Topic 1

1.1 Subfields of software engineering can be found in

a) SWEBOK YES NO

Answer 1.1 a): YES X NO

Reason: Topic 1 Slides 91 – 95:

SWEBOK is the official definition of subfields of SE in a systematic way (by a classification)

b) Textbooks on SE YES NO

Answer 1.1 b): YES X NO

Reason: Topic 1 Slides 12 & 91:

Areas of software engineering can be found in textbooks (unordered, partly overlapping, not complete). Before SWEBOK, SE was defined by textbooks and research papers

c) Conferences like ICSE YES NO

Answer 1.1 c): YES X NO

Reason: Topic 1 Slides 12 – 24 & 91:

The conferences like ICSE reflect the current state of the art in SE, providing a forum for researchers, practitioners and educators to present and discuss innovations, trends, experiences and concerns in the field. Like Textbooks, subfields of SE can be found in conferences (unordered and not complete) and they defined SE before an official definition (SWEBOK) was published.

1.2 Which of the sources mentioned in 1.1. do you prefer?

Explain why (very short answer)

Answer 1.2: SWEBOK

Reason: Topic 1 Slides 91 – 95:

Because SWEBOK is the official definition of SE knowledge areas, classified and complete, unlike textbooks and conferences, which are unordered, partly overlapping, and not complete.

1.3 Which subjects are subfields of software engineering?

a) Project management YES NO

Answer 1.3 a): YES X NO

Reason: Topic 1 Slides 11–12: Project management is listed as an area of SE in both the overview diagram and the list of SE areas from textbooks, research subjects, and conferences.

b) Programming YES NO

Answer 1.3 b): YES X NO

Reason: Topic 1 Slide 11: Programming is shown as a component of SE in the diagram, though described as ‘a comparatively small component of the software engineering field.

c) Data bases YES NO

Answer 1.3 c): YES NO X

Reason: Topic 1 Slides 9–10: Databases are presented as a separate field of computer science (‘Management of complex and structured volumes of data’), distinct from SE.

d) Complexity theory YES NO

Answer 1.3 d): YES NO X

Reason: Topic 1 Slides 9–12: Complexity theory does not appear in any listed SE subfield.

e) Software metrics YES NO

Answer 1.3 e): YES X NO

Reason: Topic 1 Slide 12: Software metrics is explicitly listed as an SE area.

f) Cost estimation YES NO

Answer 1.3 f): YES X NO

Reason: Topic 1 Slide 12: Cost estimation is explicitly listed as an area of SE from textbooks and research subjects.

g) Development tools YES NO

Answer 1.3 g): YES X NO

Reason: Topic 1 Slide 12: CASE tools are listed as an SE area. Slide 17: ‘Tools and environments’ appears as an ICSE topic.

h) Compiler construction YES NO

Answer 1.3 h): YES NO X

Reason: Topic 1 Slides 9–10: Compiler construction is presented as a separate, well-defined field of CS (‘Translation of higher-level programming languages to machine-level languages’), distinct from SE.

i) Agile software development YES NO

Answer 1.3 i): YES X NO

Reason: Topic 1 Slides 12, 17: 'Agile development' / 'Agile software development' appears both in the list of SE areas from textbooks and as a topic at ICSE conferences

j) Refactoring YES NO

Answer 1.3 j): YES X NO

Reason: Topic 1 Slides 41–44: Refactoring is presented as a SE activity for improving the design of legacy software, with a defined process using automated regression tests.

1.4 What is the fundamental subject of software engineering (a – g below)?

a) Economical development of high-quality software YES NO

Answer 1.4 a): YES X NO

Reason: Topic 1 Slides 56–57: This is Definition 1 by Pagel/Six. It is a valid definition of SE focusing on two main conflicting goals (economy and quality).

b) Development of high-quality programming languages YES NO

Answer 1.4 b): YES NO X

Reason: Topic 1 Slides 9–10, 56–57: None of the SE definitions mention programming languages. Programming language development belongs to compiler construction / programming language theory, not SE.

c) Programming to have an excellent product YES NO

Answer 1.4 c): YES NO X

Reason: Topic 1 Slides 11, 74, 76: Programming is only about 20% of the software development process in practice. SE is much broader than just programming.

d) Complexity theory to develop efficient algorithms YES NO

Answer 1.4 d): YES NO X

Reason: Topic 1 Slides 9–12: Complexity theory is not mentioned in any SE definition or SE subfield. It belongs to theoretical computer science.

e) Systematic, disciplined, quantifiable approach to the development of software YES NO

Answer 1.4 e): YES X NO

Reason: Topic 1 Slides 56–57: This is Definition 3 from the IEEE Standard Glossary of Software Engineering Terminology (1990, specialized in 2010). It is the official IEEE definition.

f) Management of complex data bases YES NO

Answer 1.4 f): YES NO X

Reason: Topic 1 Slides 9–10: Database management is explicitly defined as a separate field of computer science, not SE.

g) Problems of development of complex (large-scale) software YES NO

Answer 1.4 g): YES ☒ NO

Reason: Topic 1 Slides 56–57: This corresponds to Definition 2 by Sommerville: “SE is an engineering discipline concerned with the practical problems of developing large software systems.”

1.5 Assess the following statements

a) The field of software engineering originated in the moment when first high-level programming languages were developed, i.e. in the 1950th.

YES NO

Answer 1.5 a): YES NO ☒

Reason: Topic 1 Slides 51, 54: The first high-level programming language (Fortran) was developed in 1957, but software engineering as a discipline originated at the NATO conferences of 1968/69. These are different events separated by about a decade.

b) Software engineering as a discipline was developed as an initiative of NATO.

YES NO

Answer 1.5 b): YES ☒ NO

Reason: Topic 1 Slides 51, 53–54: The NATO conferences of 1968 and 1969 (Garmisch-Partenkirchen) introduced the notion of “software engineering.” SE as a discipline was started from this NATO initiative.

c) Expenses of maintenance are lower than or equal to the development of software.

YES NO

Answer 1.5 c): YES NO ☒

Reason: Topic 1 Slide 76: Software has long-run usage (10–25 years) with modifications after delivery. Maintenance expenses are typically higher than development costs.

d) It is assumed that expenses of coding are about 20 % of the whole software development process. This percentage (20 %) is about the same for small, medium, and large software.

YES NO

Answer 1.5 d): YES NO ☒

Reason: Topic 1 Slides 74, 76: While coding is about 20% overall, the chart on Slide 74 clearly shows that the coding percentage varies dramatically with project size — for small software coding takes around 85%, while for large software it drops to around 20%. The 20% is not the same across all sizes.

Topic 2

2.1 External vs. internal criteria

External quality criteria of software can be observed by the user of the system. Which of the following criteria are external ones?

a) Robustness YES NO

Answer 2.1 a): YES **X** NO

Reason: Topic 2 Slide 18:

Robustness is observable during execution (external).

b) Maintainability YES NO

Answer 2.1 b): YES NO **X**

Reason: Topic 2 Slide 18:

Maintainability refers to internal code changes and modifications.

c) Scalability YES NO

Answer 2.1 c): YES **X** NO

Reason: Topic 2 Slide 18:

Scalability is observed in the system's behavior under load.

d) Efficiency YES NO

Answer 2.1 d): YES **X** NO

Reason: Topic 2 Slide 18:

Efficiency (run time, space) is observable when the program is being executed.

2.2 Portability

a) Software is portable, if the effort for the transfer into another environment is significantly smaller than the according redevelopment. YES NO

Answer 2.2 a): YES **X** NO

Reason: Topic 2 Slide 10:

This is the essence of portability. The change efforts for the transfer are essentially smaller than for the new implementation.

b) Software is only portable if it can be transferred into a new environment (hardware, operating system) without any changes. YES NO

Answer 2.2 b): YES NO **X**

Reason: Topic 2 Slide 10:

Portability does not require to have the software running in other requirements without any modification.

2.3 Scalability

a) Good scalability for a program means to depend on the input (small or big) only in a linear way, i.e. depends in respond time at most linearly on the size of the input.

YES NO

Answer 2.3 a): YES **X** NO

Reason: Topic 2 Slide 12:

Good scalability implies that performance increases linearly with input size, (not exponential). Large input should be handled without loss of (linear) efficiency.

b) Good scalability for a program means to behave for big input like for small input (e.g. same respond time)

YES NO

Answer 2.3 b): YES NO **X**

Reason: Topic 2 Slide 12:

Time increases are possible with scalability, but they should be linear rather than constant.

Topic 3

3.1

What are the fundamental features of the **V-Model**?

a) In the V-model, test activities are part of the whole development process.

YES NO

3.1 a): YES **X** NO

Reason: Topic 3, slide 67:

V-Model is an improvement of the Waterfall model where testing is included during the whole development not only in the end.

b) Test cases are derived from different kinds of software documents, e.g. requirements specification, design, program units.

YES NO

3.1 b): YES **X** NO

Reason: Topic 3 Slides 67 and 70:

Test cases are derived from four different kinds of software documents (requirements specification, architectural design, detailed design and unit implementation). Each of them produces a set of test cases.

c) In the V-model, testing the whole system (acceptance testing) can be completed already at the beginning of the development since acceptance test data are derived from the requirements at the beginning.

YES NO

3.1 c): YES NO **X**

Reason: Topic 3 Slides 69:

The V-Model explains that test cases for acceptance testing are derived early from requirements but the execution of acceptance testing happens last, following the bottom up order of testing, not at the beginning.

d) The V-model has been created for large software systems to be able to test in abstraction levels.

YES NO

3.1 d): YES **X** NO

Reason: Topic 3 Slides 69-73:

V-Model is described as Waterfall model extended by testing in abstraction levels because testing happens across multiple abstraction levels e.g. unit, integration, system and acceptance.

3.2 Which of the three orders of activities are possible according to the V-model:

Order 1 YES NO

Order 2 YES NO

Order 3 YES NO

In addition to YES and NO, evaluate each order 1/2/3: best order, possible order, impossible order.

Order 1 (12 single steps):

Requirements definition

 Derive: Application scenarios

Architectural design

 Derive: Test cases (A)

Detailed design

 Derive: Test cases (D)

Unit implementation

 Derive: Test cases (U)

Unit testing

Integration testing

System testing

Acceptance testing

Order 2 (12 single steps):

Requirements definition

Architectural design

Detailed design

Unit implementation

 Derive: Application scenarios

 Derive: Test cases (A)

 Derive: Test cases (D)

 Derive: Test cases (U)

Unit testing

Integration testing

System testing

Acceptance testing

Order 3 (12 single steps):

Requirements definition

Architectural design

Detailed design
Unit implementation
 Derive: Application scenarios
Acceptance testing
 Derive: Test cases (A)
System testing
 Derive: Test cases (D)
Integration testing
 Derive: Test cases (U)
Unit testing

3.2):

Reason: Topic 3 Slides 68-72:

Order 1 : YES **X** NO

Test cases are derived in the correctlu, at each corresponding development phase.

Order 2 : YES NO **X**

Test cases are derived after implementation which is too late and not in the correct order.

Order 2 : YES NO **X**

Test cases are derived after testing has started before and the acceptance testing comes to early. This is not how V-Model works.

3.3 Which of the following statements concerning **agile methods** are true/false?

a) Agile methods in software development are not process models. They are only a collection of useful principles and practices, like close cooperation between customer and software developers, or the frequent delivery of working software.

YES NO

Answer 3.3 a): YES **X** NO

Reason: Topic 3 Slides 81 – 85:

Agile development is not a process model, it is rather a collection of principles, values and practices of project management, which can be applied together with traditional process models.

b) Examples of agile methods are Scrum and the V-model.

YES NO

Answer 3.3 b): YES NO **X**

Reason: Topic 3 Slides 66-72 & 84 & 88-90:

Scrum is an agile method, focused on project management, but V-model is an enrichment of ordinary phase models (e.g. waterfal model) to emphasize the distinguished role of testing.

c) A weak point of agile methods is often a bad software architecture

YES NO

Answer 3.3 c): YES **X** NO

Reason: Topic 3 Slides 92:

Bad software architecture is a weak point of agile methods. An undocumented software architecture is a known danger in agile projects.

d) Agile Methods have been introduced to handle large-scale (complex) software systems

YES NO

Answer 3.3 d): YES NO **X**

Reason: Topic 3 Slides 81– 93:

Agile methods were introduced for tight, self-organizing teams, and ways to craft the code and the team such that the inevitable requirements churn was not a crisis.

Topic 4

4.1 What is true/false about basic concepts in software development?

a) A basic concept is an elementary formalized description mechanism for software documents.

YES NO

Answer 4.1 a): YES **X** NO

Reason: Topic 4 Slides 4-16:

Basic concepts are precisely elementary and formalized description mechanisms. Not a natural language, not a combination of techniques like UML, but elementary and formalized mechanisms.

b) Basic concepts are necessary to overcome the difficulties of textual descriptions (e.g. misunderstandings, natural language not unique).

YES NO

Answer 4.1 b): YES **X** NO

Reason: Topic 4 Slide 4:

Basic concepts complement and support textual descriptions by providing higher accuracy.

c) Thus, basic concepts complement textual descriptions, i.e. both approaches might be necessary in the same software project.

YES NO

Answer 4.1 c): YES **X** NO

Reason: Topic 4 Slide 4:

Basic concepts are used to support and complement natural language descriptions. They both can exist in the same project.

d) UML is a basic concept since it collects a lot of different graphical representations which are strongly formalized with respect to the syntax (of that graphical notation).

YES NO

Answer 4.1 d): YES NO **X**

Reason: Topic 4 Slides 91 – 95:

UML is not a basic concept, because it gathers many elementary techniques (class diagram, sequence diagram, etc.). A basic concept must be elementary and UML is a combination, not a single element.

4.2 Classification:

a) The fundamental classification of basic concepts is based on the view on the software, e.g. view of functionality, view of the objects of the software, view of the classes of the software.

Thus, the same software can be described by many different views.

YES NO

Answer 4.2 a): YES **X** NO

Reason: Topic 4 Slides 17 – 19:

The same software can be seen from the Functional View, Data-Oriented View, Object-Oriented View, Algorithmic View, State-Oriented View, Scenario-Based View all at the same time.

b) Since each application is connected with some main view, the same software can be described only by basic concepts of this main view.

YES NO

Answer 4.2 b): YES NO **X**

Reason: Topic 4 Slide 17:

The same software can be viewed from different views and it is not limited only to the 'main view'.

4.3 Properties of basic concepts

a) Basic concepts are phase-independent which means that each basic concept can always be used in all software development phases (e.g. specification, design, coding).

YES NO

Answer 4.3 a): YES NO **X**

Reason: Topic 4 Slide 28:

Phase-independence means that basic concepts can be used in different phases, but not each basic concept can always be used in all phases.

b) Application-independence of basic concepts means that there might be some basic concepts usable in different application areas

YES NO

Answer 4.3 b): YES **X** NO

Reason: Topic 4 Slides 30 – 31:

Some basic concepts are usable in different application fields such as technical-scientific, real-time, administrative, HCI — thus they are application-independent.